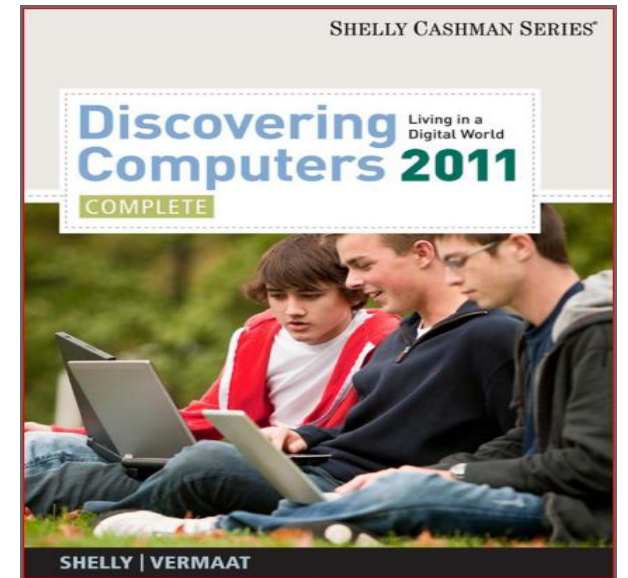


Lecture 3:



Safety and Software Development methodologies

Topic to be Covered

Real time systems characteristics (timing constraints)

Real time systems concepts; machine vision basics; Design the machine vision application; serial sensors and actuators: Digital signal processors (DSP): **Analysis and design of real time systems. Real time software design methods.** Device handling and interrupts; Signal representation; sensor physics; Rate monotonic scheduling: Biomedical instruments: **Measuring instruments**; Speech processing: Image recognition systems. Speech recognition systems; **Malt-lab image processing toolbox; malt-lab programming.**

Software Development Methodologies



No one would consider building a bridge, or a spacecraft, without using the rules for development that have become second nature to developers of hardware.



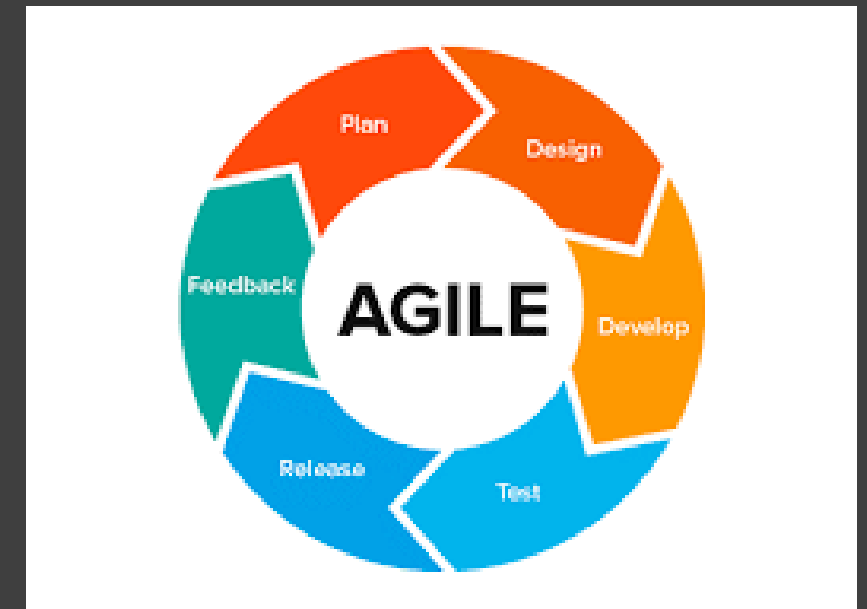
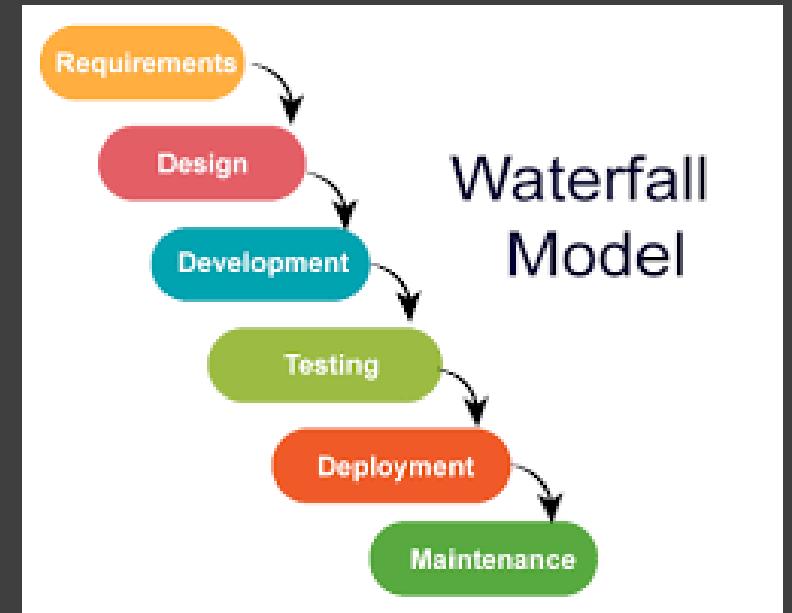
Software is flexible and “easy” to alter, it is even more important to have a disciplined and planned approach for software.

Rules for Creating Safer Software

1. Communicate
2. Have and follow good software engineering practices and procedures
3. Perform safety and development analyses
4. Incorporate appropriate software development methodologies, techniques & design features

Software Development Methodologies

- Two main schools of thoughts on development methodologies:
 - Structured processes:
 - Follows a strict plan with little to no room for going back
 - Involves planning every detail as much as possible before development starts
 - Iterative processes:
 - Making small versions of the system and keep improving on them
 - Few features first
 - Go through the development cycle several times



Software methodology Vs. Safety

- Following a software development methodology does not ensure safety
- However: *A good software development process, understood and followed, greatly increase the odds of developing safer, more reliable software.*

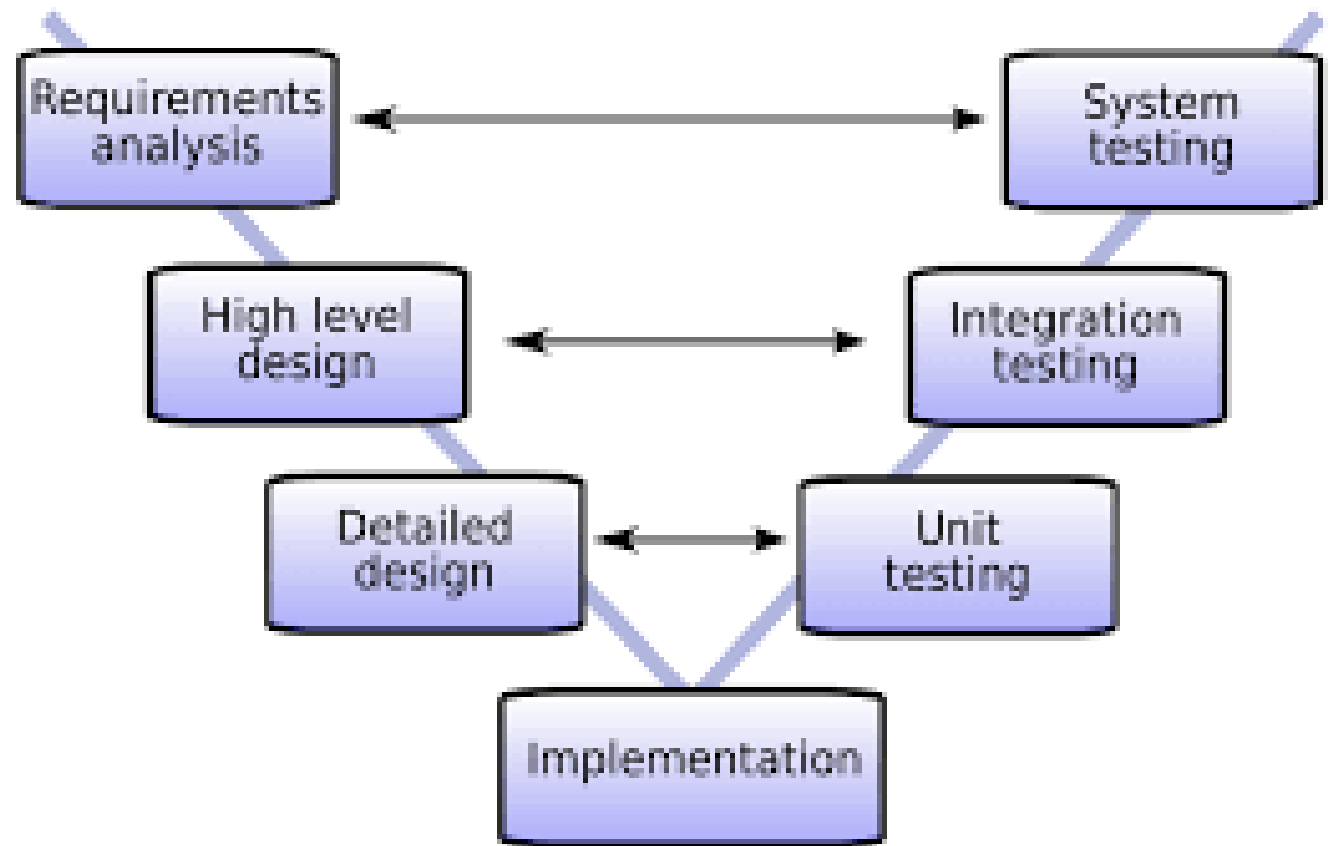
Safe methodologies?

- Regardless of the methodology selected, at the very minimum a project team must review all:
 - specifications,
 - designs,
 - implementations,
 - tests,
 - engineering change requests, and
 - problem/failure reports

to determine if any hazards have been introduced.

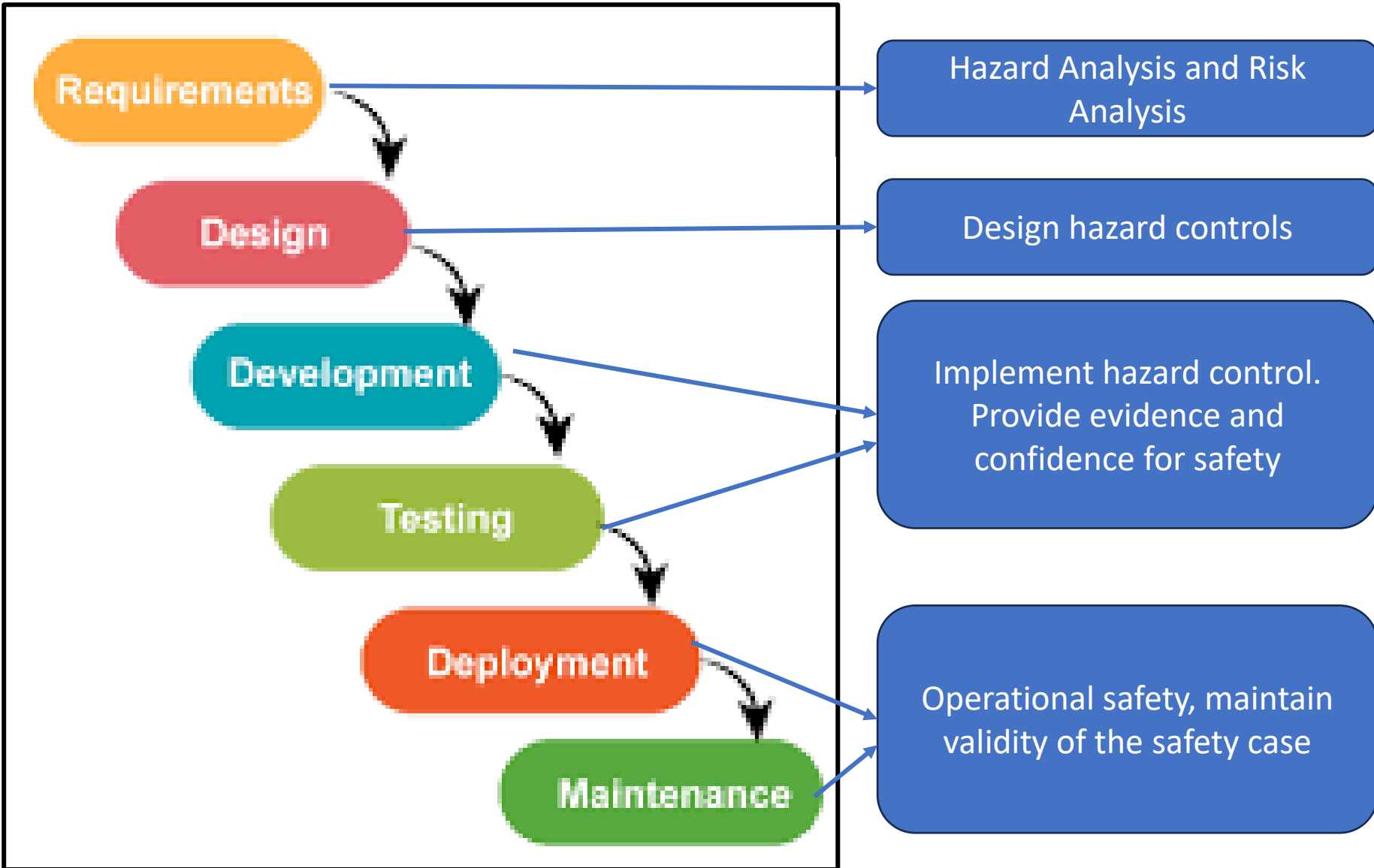
No one process or methodology is a “silver bullet.”

The V model



Safety in software development methodologies

-
- Task related to safety:
 - Perform safety analysis (hazard analysis, provide hazard reports, manage safety check lists)
 - Implement the results of the analyses. (e.g., missed safety requirements)
 - Track and verify changes



Safety Specific Development methodologies

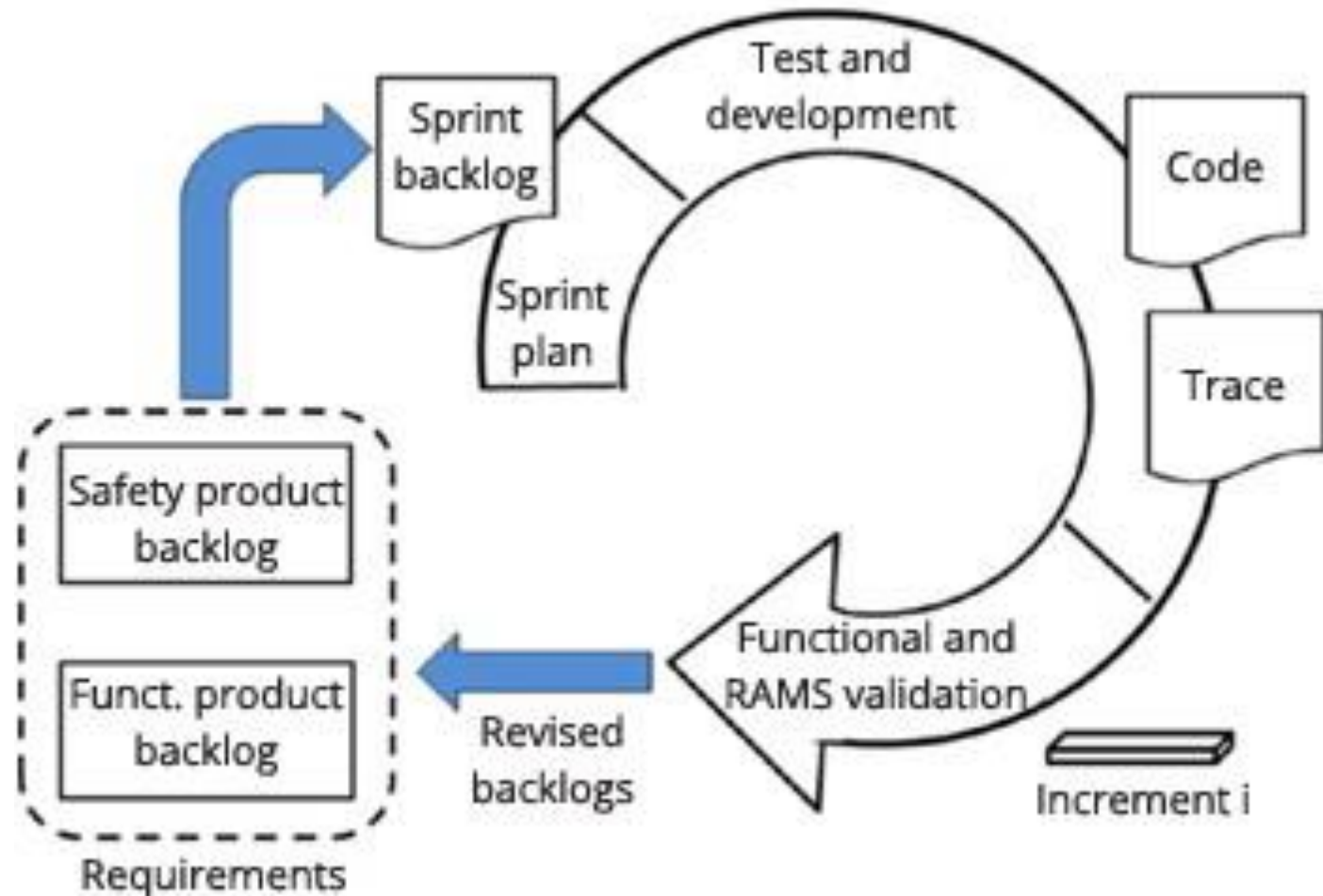
- Methodologies designed to support development of safety critical systems
- Have specific safety activities embedded in the development cycle
- Examples:
 - Safe Scrum

Scrum



Safety Specific Development methodologies

SafeScrum



Caution



- Not all the software within a project needs to be treated in an identical way.
- Safety-critical software components can have different levels of software safety effort applied to them.
- High-risk safety-critical software components may undergo a “full” safety effort, while low risk safety-critical components may only undergo the “minimum” safety effort tasks and analyses.

Discussion

-
- Examples of software that have faced safety issues due to poor safety practices in development.

Example of failure due to poor Dev practices

- Therac-25
 - Radiation therapy machine that could be controlled by a computer



Recap: Software that Kills

- Early to mid-1980s
- Revolutionary *Double-Pass* medical particle accelerator
- Moved to complete software control
- Injured 6 people, killing 3 of them
- Two different underlying bugs
- But it was more than just bugs
 - Poor software engineering practices



Software Engineering Issues in Therac-25

-
- AECL did not have the software code independently reviewed and chose to rely on in-house code, including the operating system.
 - The engineer had reused software from the Therac-6 and Therac-20, which used hardware interlocks that masked their software defects.
 - AECL had never tested the Therac-25 with the combination of software and hardware until it was assembled at the hospital.

Model Driven Development

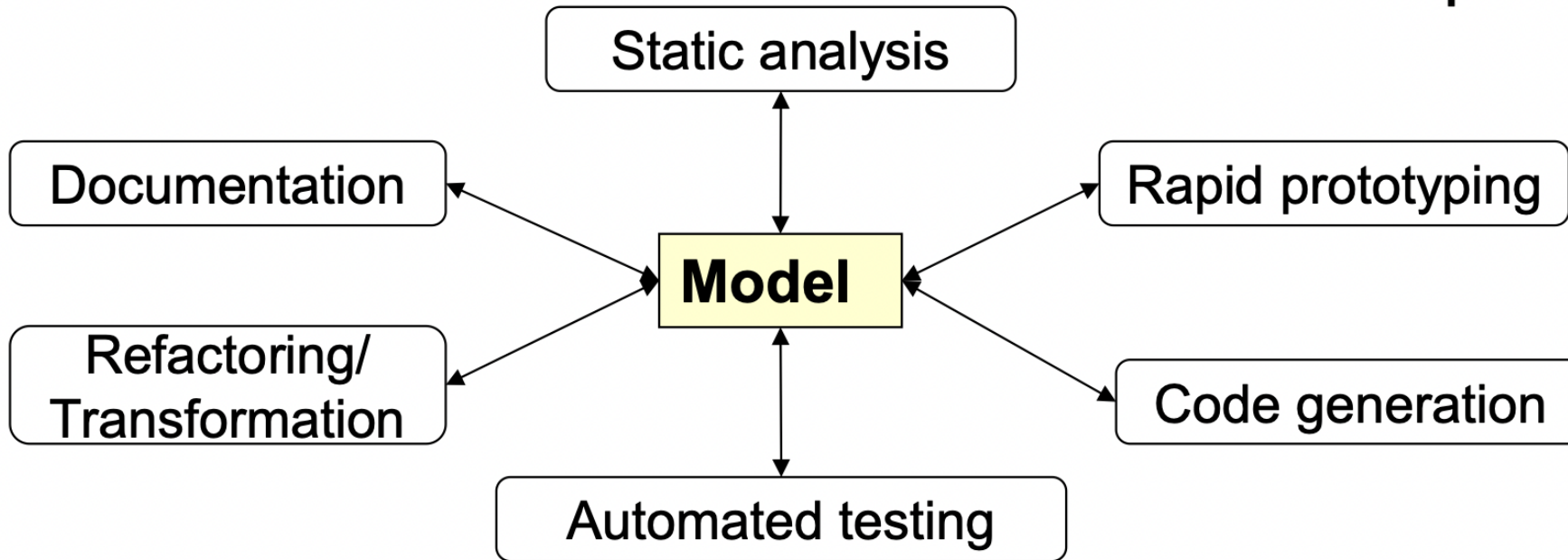
—

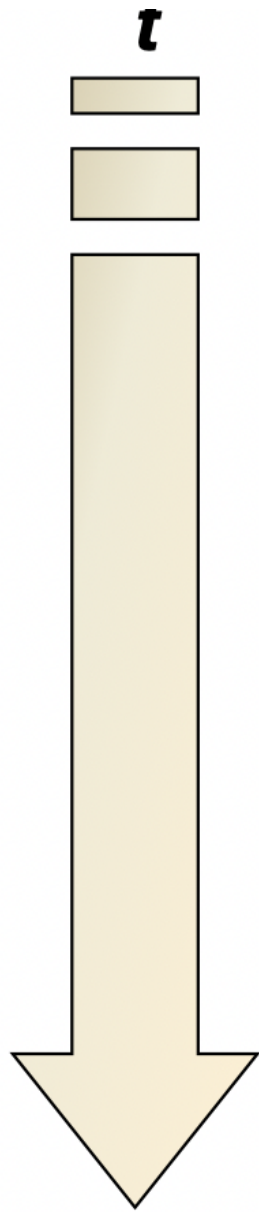
Class Background check

- Have you ever heard of MDD?
- Model-driven development (MDD) is a format to write and implement software quickly, effectively and at minimum cost.
- The methodology is also known as *model-driven software development* (MDSD), *model-driven engineering* (MDE) and *model-driven architecture* (MDA).

Model Driven Software Engineering

- Model as the **central artifact** of software development





- ***Models as drafts***

- Communication of ideas and alternatives
- Objective: modeling per se

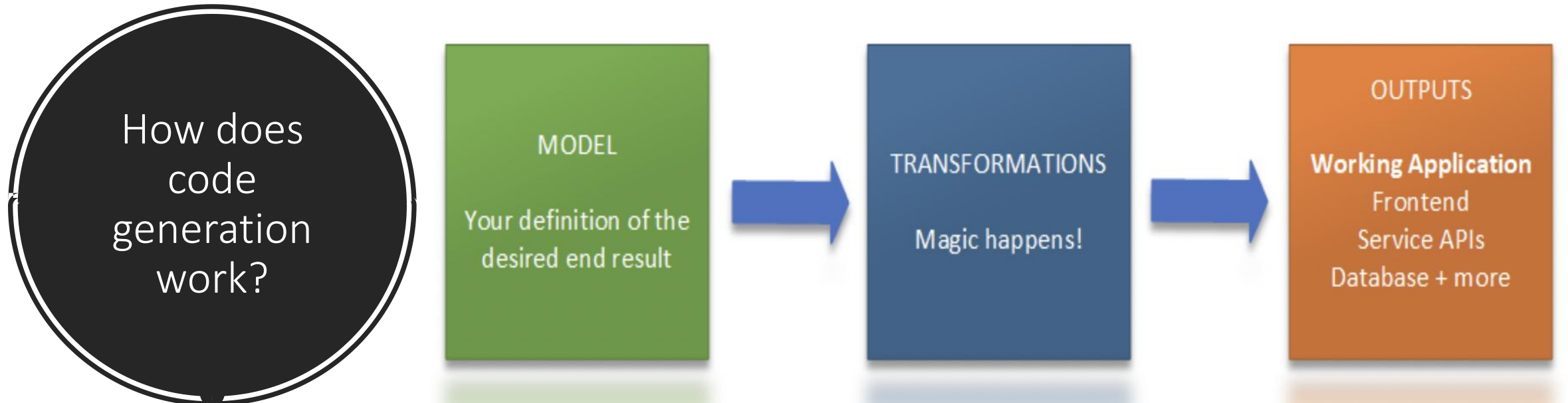
- ***Models as guidelines***

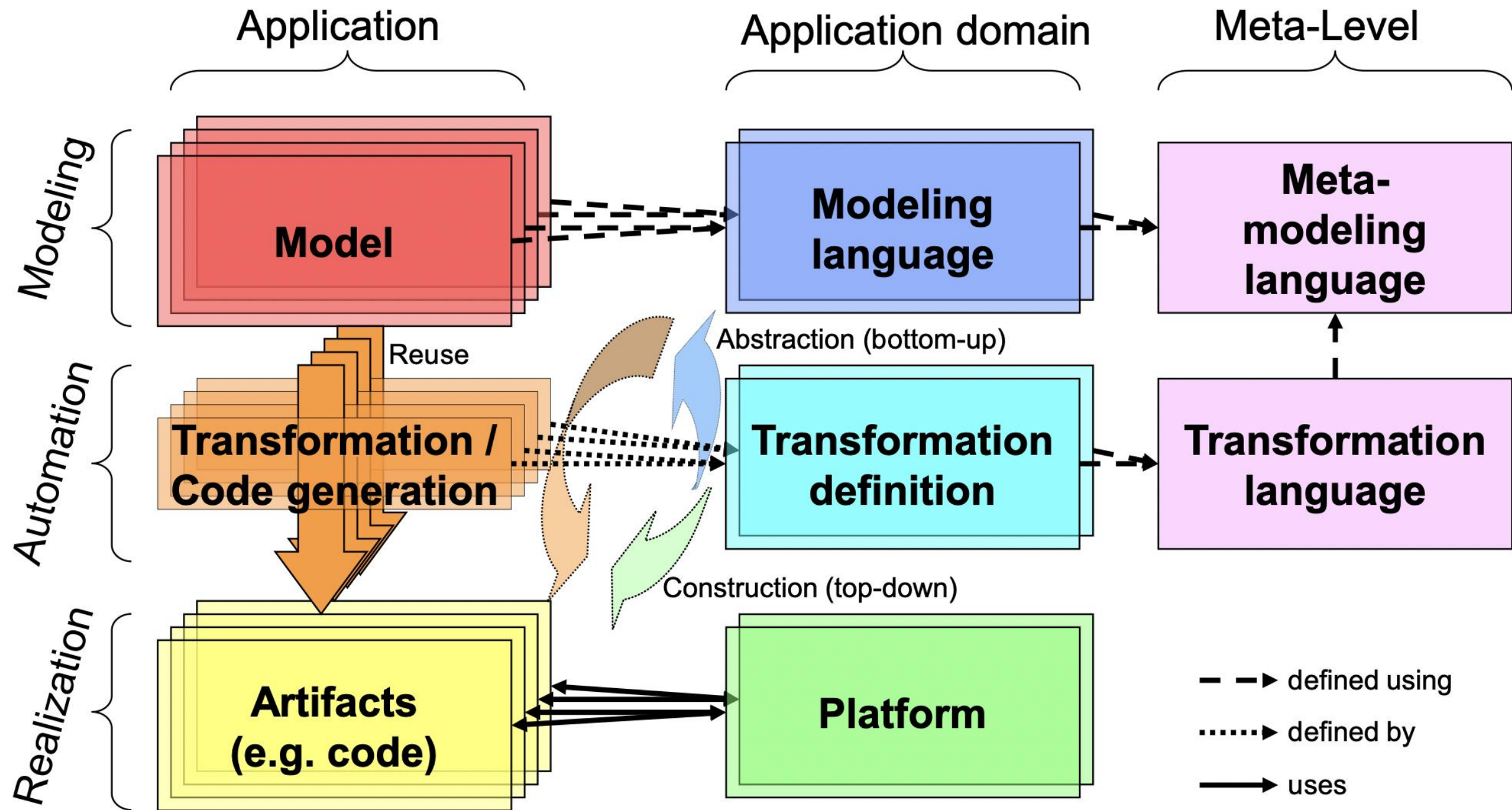
- Design decisions are documented
- Objective: instructions for implementation

- ***Models as programs***

- Applications are generated automatically
- Objective: models are source code and vice versa

Models + Transformations = Software

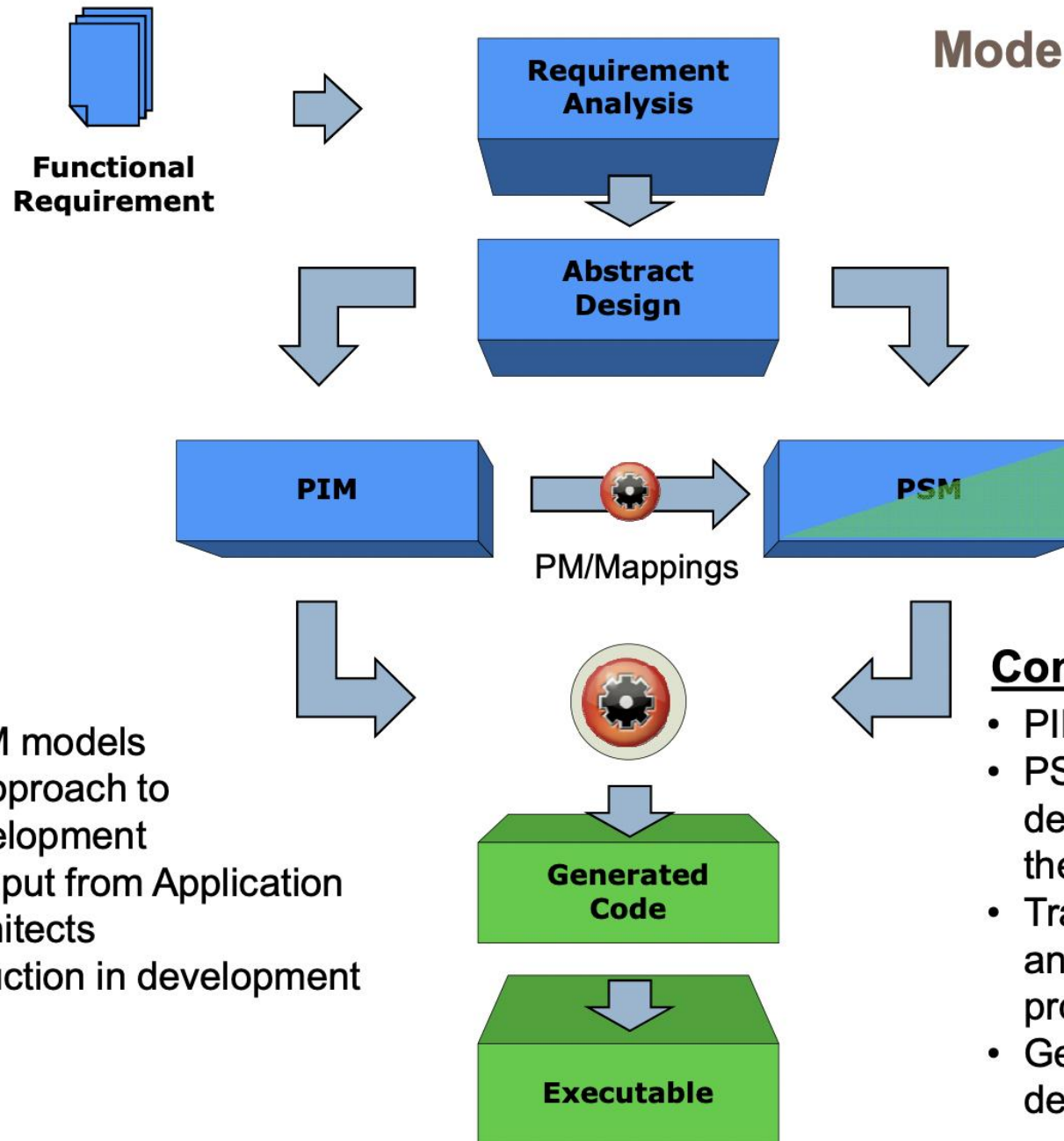




Basics of MDD

- Traditional software development methodologies:
 - Requirements→Design→Code→Unit, Integration, and System test
- Model driven software development:
 - Requirements→Model→Verify (test) and Debug→Generate code→System test”.

Model Driven Architecture (MDA)



Pros

- Reusable PIM models
- Methodical approach to software development
- More direct input from Application Analysts/Architects
- Marginal reduction in development time

Cons

- PIM models cannot be validated
- PSM are platform/architecture dependent, which makes up for the majority of the artifacts
- Transformation/code-generation is an integral part of the development process
- Generated code still require further development

MDD for safety critical systems

-
- Carefully designing the models increases reliability of the system
 - Models can be used to **test** the system before implementation
 - Test for concurrency issues, timing issues, deadlock etc.
 - Code generation reduces developer errors

CODE Generation

Library
- books: Book[]
+ addBook()
+ removeBook()
+searchBook()

```
// Library.java
public class Library {
    private Book[] books;

    public void addBook(Book book) {
        // Add book to the library
    }

    public void removeBook(Book book) {
        // Remove book from the library
    }

    public Book[] searchBook(String keyword) {
        // Search books by keyword
        return null;
    }
}
```

Code Generation Demo

- Install Eclipse Modelling Framework – most basic framework
- Create a class diagram
- Generate code!

MDE in real life applications

- Custom made modelling languages
- Custom made transformations

Questions

